

Prefix-Tuning: Optimizing Continuous Prompts for Generation

Xiang Lisa Li
Stanford University
xlisali@stanford.edu

Percy Liang
Stanford University
pliang@cs.stanford.edu

Abstract

Fine-tuning is the de facto way to leverage large pretrained language models to perform downstream tasks. However, it modifies all the language model parameters and therefore necessitates storing a full copy for each task. In this paper, we propose prefix-tuning, a lightweight alternative to fine-tuning for natural language generation tasks, which keeps language model parameters frozen, but optimizes a small *continuous task-specific* vector (called the prefix). Prefix-tuning draws inspiration from prompting, allowing subsequent tokens to attend to this prefix as if it were “virtual tokens”. We apply prefix-tuning to GPT-2 for table-to-text generation and to BART for summarization. We find that by learning only 0.1% of the parameters, prefix-tuning obtains comparable performance in the full data setting, outperforms fine-tuning in low-data settings, and extrapolates better to examples with topics unseen during training.

1 Introduction

Fine-tuning is the prevalent paradigm for using large pretrained language models (LMs) (Radford et al., 2019; Devlin et al., 2019) to perform downstream tasks (e.g., summarization), but it requires updating and storing all the parameters of the LM. Consequently, to build and deploy NLP systems that rely on large pretrained LMs, one currently needs to store a modified copy of the LM parameters for each task. This can be prohibitively expensive, given the large size of current LMs; for example, GPT-2 has 774M parameters (Radford et al., 2019) and GPT-3 has 175B parameters (Brown et al., 2020).

A natural approach to this problem is *lightweight fine-tuning*, which freezes most of the pretrained parameters and augments the model with small trainable modules. For example, adapter-tuning

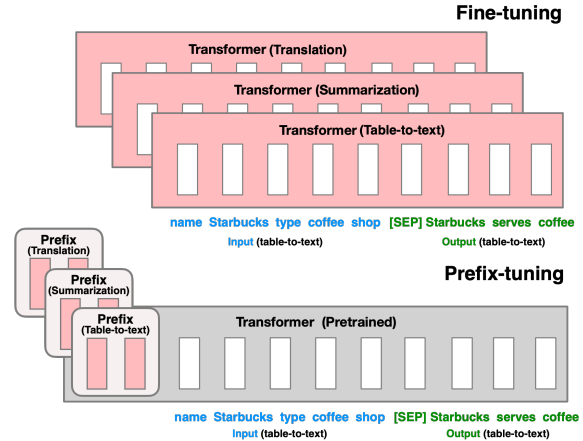


Figure 1: Fine-tuning (top) updates all Transformer parameters (the red Transformer box) and requires storing a full model copy for each task. We propose prefix-tuning (bottom), which freezes the Transformer parameters and only optimizes the prefix (the red prefix blocks). Consequently, we only need to store the prefix for each task, making prefix-tuning modular and space-efficient. Note that each vertical block denote transformer activations at one time step.

(Rebuffi et al., 2017; Houlsby et al., 2019) inserts additional task-specific layers between the layers of pretrained language models. Adapter-tuning has promising performance on natural language understanding and generation benchmarks, attaining comparable performance with fine-tuning while adding only around 2-4% task-specific parameters (Houlsby et al., 2019; Lin et al., 2020).

On the extreme end, GPT-3 (Brown et al., 2020) can be deployed without any task-specific tuning. Instead, users prepend a natural language task instruction (e.g., *TL;DR* for summarization) and a few examples to the task input; then generate the output from the LM. This approach is known as in-context learning or *prompting*.

In this paper, we propose *prefix-tuning*, a lightweight alternative to fine-tuning for natural language generation (NLG) tasks, inspired by prompting. Consider the task of generating a textual de-

scription of a data table, as shown in Figure 1, where the task input is a linearized table (e.g., “name: Starbucks | type: coffee shop”) and the output is a textual description (e.g., “Starbucks serves coffee.”). Prefix-tuning prepends a sequence of *continuous task-specific* vectors to the input, which we call a *prefix*, depicted by red blocks in Figure 1 (bottom). For subsequent tokens, the Transformer can attend to the prefix as if it were a sequence of “virtual tokens”, but unlike prompting, the prefix consists entirely of free parameters which do not correspond to real tokens. In contrast to fine-tuning in Figure 1 (top), which updates all Transformer parameters and thus requires storing a tuned copy of the model for each task, prefix-tuning only optimizes the prefix. Consequently, we only need to store one copy of the large Transformer and a learned task-specific prefix, yielding a very small overhead for each additional task (e.g., 250K parameters for table-to-text).

In contrast to fine-tuning, prefix-tuning is modular: we train an upstream prefix which steers a downstream LM, which remains unmodified. Thus, a single LM can support many tasks at once. In the context of personalization where the tasks correspond to different users (Shokri and Shmatikov, 2015; McMahan et al., 2016), we could have a separate prefix for each user trained only on that user’s data, thereby avoiding data cross-contamination. Moreover, the prefix-based architecture enables us to even process examples from multiple users/tasks in a single batch, something that is not possible with other lightweight fine-tuning approaches.

We evaluate prefix-tuning on table-to-text generation using GPT-2 and abstractive summarization using BART. In terms of storage, prefix-tuning stores 1000x fewer parameters than fine-tuning. In terms of performance when trained on full datasets, prefix-tuning and fine-tuning are comparable for table-to-text (§6.1), while prefix-tuning suffers a small degradation for summarization (§6.2). In low-data settings, prefix-tuning on average outperforms fine-tuning on both tasks (§6.3). Prefix-tuning also extrapolates better to tables (for table-to-text) and articles (for summarization) with unseen topics (§6.4).

2 Related Work

Fine-tuning for natural language generation. Current state-of-the-art systems for natural language generation are based on fine-tuning pre-

trained LMs. For table-to-text generation, Kale (2020) fine-tunes a sequence-to-sequence model (T5; Raffel et al., 2020). For extractive and abstractive summarization, researchers fine-tune masked language models (e.g., BERT; Devlin et al., 2019) and encode-decoder models (e.g., BART; Lewis et al., 2020) respectively (Zhong et al., 2020; Liu and Lapata, 2019; Raffel et al., 2020). For other conditional NLG tasks such as machine translation and dialogue generation, fine-tuning is also the prevalent paradigm (Zhang et al., 2020c; Stickland et al., 2020; Zhu et al., 2020; Liu et al., 2020). In this paper, we focus on table-to-text using GPT-2 and summarization using BART, but prefix-tuning can be applied to other generation tasks and pre-trained models.

Lightweight fine-tuning. Lightweight fine-tuning freezes most of the pretrained parameters and modifies the pretrained model with small trainable modules. The key challenge is to identify high-performing architectures of the modules and the subset of pretrained parameters to tune. One line of research considers removing parameters: some model weights are ablated away by training a binary mask over model parameters (Zhao et al., 2020; Radiya-Dixit and Wang, 2020). Another line of research considers inserting parameters. For example, Zhang et al. (2020a) trains a “side” network that is fused with the pretrained model via summation; adapter-tuning inserts task-specific layers (adapters) between each layer of the pretrained LM (Houlsby et al., 2019; Lin et al., 2020; Rebuffi et al., 2017; Pfeiffer et al., 2020). Compared to this line of work, which tunes around 3.6% of the LM parameters, our method obtains a further 30x reduction in task-specific parameters, tuning only 0.1% while maintaining comparable performance.

Prompting. Prompting means prepending instructions and a few examples to the task input and generating the output from the LM. GPT-3 (Brown et al., 2020) uses manually designed prompts to adapt its generation for different tasks, and this framework is termed *in-context learning*. However, since Transformers can only condition on a bounded-length context (e.g., 2048 tokens for GPT-3), in-context learning is unable to fully exploit training sets longer than the context window. Sun and Lai (2020) also prompt by keywords to control for sentiment or topic of the generated sentence. In natural language understanding tasks, prompt

engineering has been explored in prior works for models like BERT and RoBERTa (Liu et al., 2019; Jiang et al., 2020; Schick and Schütze, 2020). For example, AutoPrompt (Shin et al., 2020) searches for a sequence of discrete trigger words and concatenates it with each input to elicit sentiment or factual knowledge from a masked LM. In contrast with AutoPrompt, our method optimizes continuous prefixes, which are more expressive (§7.2); moreover, we focus on language generation tasks.

Continuous vectors have been used to steer language models; for example, Subramani et al. (2020) showed that a pretrained LSTM language model can reconstruct arbitrary sentences by optimizing a continuous vector for each sentence, making the vector *input-specific*. In contrast, prefix-tuning optimizes a *task-specific* prefix that applies to all instances of that task. As a result, unlike the previous work whose application is limited to sentence reconstruction, prefix-tuning can be applied to NLG tasks.

Controllable generation. Controllable generation aims to steer a pretrained language model to match a sentence level attribute (e.g., positive sentiment or topic on sports). Such control can happen at training time: Keskar et al. (2019) pretrains the language model (CTRL) to condition on metadata such as keywords or URLs. Additionally, the control can happen at decoding time, by weighted decoding (GeDi, Krause et al., 2020) or iteratively updating the past activations (PPLM, Dathathri et al., 2020). However, there is no straightforward way to apply these controllable generation techniques to enforce fine-grained control over generated contents, as demanded by tasks like table-to-text and summarization.

3 Problem Statement

Consider a conditional generation task where the input is a context x and the output y is a sequence of tokens. We focus on two tasks, shown in Figure 2 (right): In table-to-text, x corresponds to a linearized data table and y is a textual description; in summarization, x is an article and y is a short summary.

3.1 Autoregressive LM

Assume we have an autoregressive language model $p_\phi(y | x)$ based on the Transformer (Vaswani et al., 2017) architecture (e.g., GPT-2; Radford et al.,

2019) and parametrized by ϕ . As shown in Figure 2 (top), let $z = [x; y]$ be the concatenation of x and y ; let $X_{\text{id}x}$ denote the sequence of indices that corresponds to x , and $Y_{\text{id}x}$ denote the same for y .

The activation at time step i is $h_i \in \mathbb{R}^d$, where $h_i = [h_i^{(1)}; \dots; h_i^{(n)}]$ is a concatenation of all activation layers at this time step, and $h_i^{(j)}$ is the activation of the j -th Transformer layer at time step i .¹

The autoregressive Transformer model computes h_i as a function of z_i and the past activations in its left context, as follows:

$$h_i = \text{LM}_\phi(z_i, h_{<i}), \quad (1)$$

where the last layer of h_i is used to compute the distribution for the next token: $p_\phi(z_{i+1} | h_{\leq i}) = \text{softmax}(W_\phi h_i^{(n)})$ and W_ϕ is a pretrained matrix that map $h_i^{(n)}$ to logits over the vocabulary.

3.2 Encoder-Decoder Architecture

We can also use an encoder-decoder architecture (e.g., BART; Lewis et al., 2020) to model $p_\phi(y | x)$, where x is encoded by the bidirectional encoder, and the decoder predicts y autoregressively (conditioned on the encoded x and its left context). We use the same indexing and activation notation, as shown in Figure 2 (bottom). h_i for all $i \in X_{\text{id}x}$ is computed by the bidirectional Transformer encoder; h_i for all $i \in Y_{\text{id}x}$ is computed by the autoregressive decoder using the same equation (1).

3.3 Method: Fine-tuning

In the fine-tuning framework, we initialize with the pretrained parameters ϕ . Here p_ϕ is a trainable language model distribution and we perform gradient updates on the following log-likelihood objective:

$$\max_{\phi} \log p_\phi(y | x) = \sum_{i \in Y_{\text{id}x}} \log p_\phi(z_i | h_{<i}). \quad (2)$$

4 Prefix-Tuning

We propose prefix-tuning as an alternative to fine-tuning for conditional generation tasks. We first provide intuition in §4.1 before defining our method formally in §4.2.

¹ $h_i^{(n)}$ is composed of a key-value pair. In GPT-2, the dimension of each key and value is 1024.